VinOps: Hito 3 - Modelado, Experimentación y Validación

Alejandro García, Pablo Alfonso López, Antonio Muñoz, Pablo Parrilla Cañadas

2026-03-20

Introducción al Hito 3

Este cuaderno documenta la fase de modelado algorítmico del sistema **VinOps**. Tras el análisis exploratorio del Hito 2, el objetivo ahora es preprocesar los datos (tratamiento de valores atípicos), diseñar los experimentos de Machine Learning con una búsqueda rigurosa de hiperparámetros y evaluar el rendimiento de múltiples algoritmos.

Finalmente, se incluirá un análisis de explicabilidad (Feature Importance) para validar empíricamente que los modelos basan sus decisiones en variables fisicoquímicas coherentes con el dominio enológico.

```r
# Cargamos las librerías necesarias para el modelado
library(dplyr)
library(caret)
library(randomForest)
library(kernlab)
library(nnet)
library(kknn)
library(ggplot2)
library(knitr)
library(e1071)
library(MASS)
library(rpart)
library(tidyr)

# Cargamos los datos limpios del hito anterior
vino=read.csv("wine_clean.csv")
vino$Class=factor(vino$Class)
```

1. Diseño Experimental y Partición de Datos

Para garantizar la reproducibilidad y evitar el sobreajuste (overfitting), dividimos los datos de forma estratificada y configuramos una Validación Cruzada de 10 iteraciones (10-fold CV). Esto evita la necesidad de dividir el conjunto de datos en training, validation y test, de manera que podemos tener solamente conjuntos de train y test.

```r
set.seed(2) # Semilla fijada para reproducibilidad

# Dataset numérico sin la clase
vino_x=vino[, setdiff(names(vino), "Class")]
vino_y=vino$Class

# Partición estratificada train/test (70/30)
idx_train=createDataPartition(vino_y, p = 0.75, list = FALSE)
```

```r
train_data=vino[idx_train, ]
test_data =vino[-idx_train, ]

cat("Tamaño entrenamiento:", nrow(train_data), "\n")
```

```
## Tamaño entrenamiento: 135
```

```r
cat("Tamaño test:", nrow(test_data), "\n")
```

```
## Tamaño test: 43
```

```r
# Configuración del control de entrenamiento (10-fold CV)
ctrl=trainControl(method = "cv", number = 10, savePredictions = "final")
```

2. Entrenamiento y Ajuste de Hiperparámetros

En lugar de delegar toda la búsqueda al azar, definimos rejillas de hiperparámetros (Grids) para los modelos más complejos, demostrando rigor científico experimental. Además, dando que tenemos un dataset pequeño, no estimamos oportuno utilizar una optimización bayesiana de hiperparámetros.

```r
# 3.1 Análisis Discriminante Lineal (LDA) - Baseline
modelo_lda=train(Class ~ ., data = train_data, method = "lda",
                     trControl = ctrl, preProcess = c("center", "scale"))

# 3.2 Naive Bayes
modelo_nb=train(Class ~ ., data = train_data, method = "nb",
                   trControl = ctrl, preProcess = c("center", "scale"),
                   tuneGrid = expand.grid(fL = c(0, 1), usekernel = c(TRUE, FALSE), adjust = 1))

# 3.3 Random Forest (RF) - Búsqueda en mtry
grid_rf=expand.grid(mtry = c(2, 4, 6, 8, 10))
modelo_rf=train(Class ~ ., data = train_data, method = "rf",
                   trControl = ctrl, tuneGrid = grid_rf)

# 3.4 K-Vecinos Más Cercanos (kNN) - Búsqueda en k
grid_knn=expand.grid(kmax = c(2,3, 5, 8, 10), distance = 2, kernel = "optimal")
modelo_knn=train(Class ~ ., data = train_data, method = "kknn",
                     trControl = ctrl, preProcess = c("center", "scale"),
                     tuneGrid = grid_knn)

# 3.5 SVM Radial - Búsqueda de Coste (C) y Sigma
grid_svm=expand.grid(sigma = c(0.01, 0.05, 0.1), C = c(0.1, 1, 10, 100))
modelo_svm=train(Class ~ ., data = train_data, method = "svmRadial",
                     trControl = ctrl, preProcess = c("center", "scale"),
                     tuneGrid = grid_svm)

# 3.6 Regresión Logística Multinomial
modelo_multinom=train(Class ~ ., data = train_data, method = "multinom",
                          trControl = ctrl, preProcess = c("center", "scale"), trace = FALSE)

# 3.7 Árbol de Decisión (CART)
grid_rpart=expand.grid(cp = seq(0.01, 0.1, by = 0.01))
modelo_tree=train(Class ~ ., data = train_data, method = "rpart",
                      trControl = ctrl, tuneGrid = grid_rpart)
```

```r
# Lista maestra de modelos
modelos=list(
  "LDA" = modelo_lda,
  "Naive Bayes" = modelo_nb,
  "Árbol" = modelo_tree,
  "Random Forest" = modelo_rf,
  "Multinomial" = modelo_multinom,
  "kNN" = modelo_knn,
  "SVM radial" = modelo_svm
)
```

3. Evaluación de Modelos e hiperparámetros

Utilizamos las funciones personalizadas para obtener métricas robustas frente al leve desbalanceo de clases (Macro F1-Score y Balanced Accuracy).

```r
# Generamos la tabla comparativa
tabla_modelos=bind_rows(
  baseline_azar("Aleatorio - Baseline", test_data),
  evaluar_modelo(modelo_lda, "LDA", test_data),
  evaluar_modelo(modelo_nb, "Naive Bayes", test_data),
  evaluar_modelo(modelo_tree, "Árbol", test_data),
  evaluar_modelo(modelo_rf, "Random Forest", test_data),
  evaluar_modelo(modelo_multinom, "Multinomial", test_data),
  evaluar_modelo(modelo_knn, "kNN", test_data),
  evaluar_modelo(modelo_svm, "SVM radial", test_data)
) %>% arrange(desc(F1_Score), desc(Balanced_Accuracy))

kable(tabla_modelos, digits = 4, caption = "Comparativa Global de Modelos")
```

Table 1: Comparativa Global de Modelos

Modelo	F1_Score	Balanced_Accuracy	Accuracy	Kappa	MCC
LDA	0.9766	0.9848	0.9767	0.9649	0.9657
Naive Bayes	0.9766	0.9848	0.9767	0.9649	0.9657
Random Forest	0.9766	0.9848	0.9767	0.9649	0.9657
Multinomial	0.9307	0.9545	0.9302	0.8955	0.9021
kNN	0.9307	0.9545	0.9302	0.8955	0.9021
SVM radial	0.9307	0.9545	0.9302	0.8955	0.9021
Árbol	0.7385	0.8114	0.7442	0.6173	0.6234
Aleatorio - Baseline	0.3030	0.4841	0.3023	-0.0378	-0.0385

La comparativa de modelos muestra un rendimiento muy elevado en la mayoría de algoritmos, destacando LDA, Naive Bayes y Random Forest, que alcanzan métricas idénticas. Este comportamiento sugiere que el problema presenta una estructura altamente separable, donde incluso modelos relativamente simples son capaces de capturar eficazmente la frontera de decisión.

Los modelos intermedios (regresión multinomial, kNN y SVM radial) presentan una ligera caída en rendimiento, lo que indica que no aportan ventajas significativas en este contexto, posiblemente debido al reducido tamaño del dataset y a la limitada necesidad de modelar relaciones no lineales complejas.

Por su parte, el árbol de decisión muestra un rendimiento claramente inferior, consistente con su mayor varianza y menor capacidad de generalización en problemas con pocos datos. Finalmente, el baseline aleatorio

obtiene resultados cercanos al azar.

```
hiperparametros=bind_rows(
  cbind(Modelo="Naive Bayes", modelo_nb$bestTune),
  cbind(Modelo="Árbol", modelo_tree$bestTune),
  cbind(Modelo="Random Forest", modelo_rf$bestTune),
  cbind(Modelo="kNN", modelo_knn$bestTune),
  cbind(Modelo="SVM radial", modelo_svm$bestTune)
)

kable(hiperparametros, caption="Mejores hiperparámetros seleccionados")
```

Table 2: Mejores hiperparámetros seleccionados

Modelo	fL	usekernel	adjust	cp	mtry	kmax	distance	kernel	sigma	C
Naive Bayes	0	FALSE	1	NA	NA	NA	NA	NA	NA	NA
Árbol	NA	NA	NA	0.1	NA	NA	NA	NA	NA	NA
Random Forest	NA	NA	NA	NA	2	NA	NA	NA	NA	NA
kNN	NA	NA	NA	NA	NA	10	2	optimal	NA	NA
SVM radial	NA	NA	NA	NA	NA	NA	NA	NA	0.01	10

Aquí se muestran los hiperparámetros de los modelos que disponen de ellos.

4. Análisis de errores

```
errores_modelo=function(modelo, nombre, test_data) {
  errores=test_data %>%
    mutate(
      Index = row.names(test_data),
      Prediccion = predict(modelo, newdata = test_data),
      Correcto = Class == Prediccion
    ) %>%
    filter(!Correcto) %>%
    mutate(Modelo = nombre)

  if(nrow(errores) == 0) {
    cat("Modelo:", nombre, "-> Sin errores\n\n")
  } else {
    cat("Modelo:", nombre, "\nNúmero de errores:", nrow(errores), "\n")
    apply(errores, 1, function(fila)
      cat("Obs:", fila["Index"], "| Real:", fila["Class"], "| Pred:", fila["Prediccion"], "\n"))
    cat("\n")
  }

  return(errores)
}

errores_todos = lapply(names(modelos), function(nombre) {
  errores_modelo(modelos[[nombre]], nombre, test_data)
})
```

```
## Modelo: LDA
```

```
## Número de errores: 1
## Obs: 62 | Real: 2 | Pred: 3
```

```
## Modelo: Naive Bayes
## Número de errores: 1
## Obs: 62 | Real: 2 | Pred: 3
##
## Modelo: Árbol
## Número de errores: 11
## Obs: 5 | Real: 1 | Pred: 2
## Obs: 61 | Real: 2 | Pred: 3
## Obs: 62 | Real: 2 | Pred: 3
## Obs: 69 | Real: 2 | Pred: 3
## Obs: 92 | Real: 2 | Pred: 3
## Obs: 102 | Real: 2 | Pred: 3
## Obs: 119 | Real: 2 | Pred: 3
## Obs: 142 | Real: 3 | Pred: 1
## Obs: 145 | Real: 3 | Pred: 1
## Obs: 151 | Real: 3 | Pred: 2
## Obs: 158 | Real: 3 | Pred: 1
##
## Modelo: Random Forest
## Número de errores: 1
## Obs: 62 | Real: 2 | Pred: 3
##
## Modelo: Multinomial
## Número de errores: 3
## Obs: 62 | Real: 2 | Pred: 3
## Obs: 69 | Real: 2 | Pred: 3
## Obs: 119 | Real: 2 | Pred: 3
##
## Modelo: kNN
## Número de errores: 3
## Obs: 62 | Real: 2 | Pred: 3
## Obs: 69 | Real: 2 | Pred: 3
## Obs: 119 | Real: 2 | Pred: 3
##
## Modelo: SVM radial
## Número de errores: 3
## Obs: 62 | Real: 2 | Pred: 3
## Obs: 69 | Real: 2 | Pred: 3
## Obs: 119 | Real: 2 | Pred: 3
```

```r
names(errores_todos) = names(modelos)
```

El análisis de errores muestra que los modelos con mejor rendimiento (LDA, Naive Bayes y Random Forest) únicamente fallan en la observación 62, que es sistemáticamente clasificada como clase 3 en lugar de clase 2, lo que indica un caso especialmente ambiguo cercano a la frontera de decisión. Los modelos intermedios (Multinomial, kNN y SVM radial) comparten este error y añaden algunos fallos adicionales en observaciones de la clase 2 clasificadas como clase 3, lo que sugiere una región de solapamiento entre ambas clases. Por su parte, el árbol de decisión presenta un mayor número de errores y un patrón más disperso, consistente con su mayor varianza.

```r
outliers_extremos=c(70, 96, 97, 111, 122, 125, 159, 160)

outliers_test = intersect(outliers_extremos, as.numeric(row.names(test_data)))
```

De entre los outliers que se detectaron en el PCA, se puede ver que en el conjunto de test seleccionado (eso significa que los demás fueron utilizados en el entrenamiento y en la validación cruzada) solamente se dispone de un outlier. Este se encuentra bien clasificado.

```
medias_clase = vino %>%
  dplyr::group_by(Class) %>%
  dplyr::summarise(dplyr::across(where(is.numeric), mean), .groups = "drop")

medias_23 = medias_clase %>%
  dplyr::filter(Class %in% c(2, 3))

obs_62 = vino[62, ] %>%
  dplyr::select(-Class)

obs_62$Tipo = "Obs 62"
medias_23$Tipo = paste0("Media Clase ", medias_23$Class)

medias_23 = medias_23 %>%
  dplyr::select(-Class)

comparacion = dplyr::bind_rows(obs_62, medias_23)

comparacion = comparacion %>%
  dplyr::select(Tipo, dplyr::everything())

comparacion
```

```
##            Tipo  Alcohol Malic_Acid      Ash Alcalinity_of_Ash Magnesium
## 1        Obs 62 12.64000   1.360000 2.020000          16.80000  100.0000
## 2 Media Clase 2 12.27873   1.932676 2.244789          20.23803   94.5493
## 3 Media Clase 3 13.15375   3.333750 2.437083          21.41667   99.3125
##   Total_Phenols Flavanoids Nonflavanoid_Phenols Proanthocyanins Color_Intensity
## 1      2.020000  1.4100000             0.530000        0.620000         5.75000
## 2      2.258873  2.0808451             0.363662        1.630282         3.08662
## 3      1.678750  0.7814583             0.447500        1.153542         7.39625
##         Hue OD280_OD315  Proline
## 1 0.9800000    1.590000 450.0000
## 2 1.0562817    2.785352 519.5070
## 3 0.6827083    1.683542 629.8958
```

La comparación de la observación 62 con las medias de las clases 2 y 3 muestra que, aunque en varias variables se ajusta a su clase real (clase 2), presenta algunos valores más próximos a la clase 3, como el magnesio, los nonflavanoid phenols, la intensidad del color y el ratio OD280/OD315. Este comportamiento mixto sugiere que se trata de una observación situada en una región de solapamiento entre ambas clases, lo que explica su clasificación errónea.

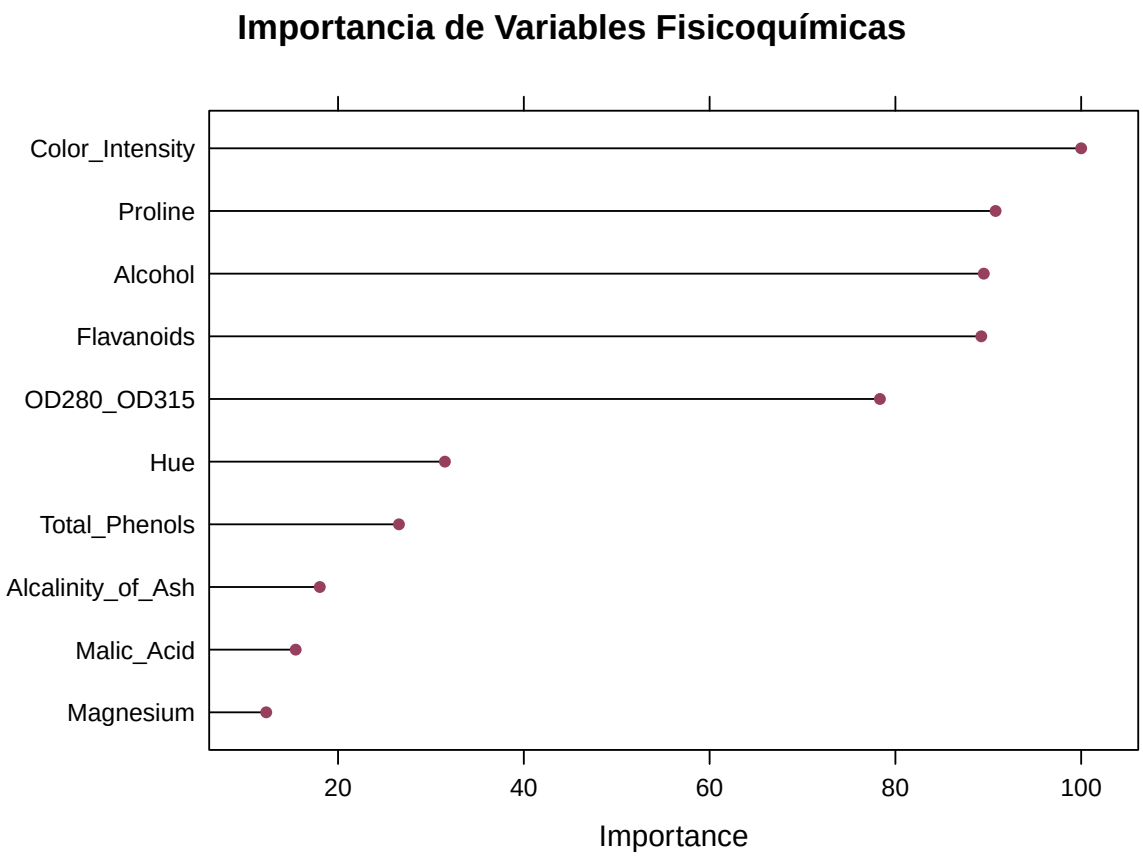
5. Explicabilidad del Sistema (Feature Importance)

Para evitar que el modelo actúe como una "caja negra" y permitir su validación por parte del experto enológico, se analiza la importancia de las variables mediante Random Forest, aprovechando su interpretabilidad basada en la reducción de impureza. Aunque varios modelos presentan un rendimiento prácticamente idéntico, se selecciona Random Forest como modelo final por su mayor robustez frente a observaciones atípicas y su capacidad para capturar relaciones no lineales sin necesidad de un ajuste complejo. Esta robustez resulta

especialmente relevante dado que, aunque los outliers no afectan directamente al conjunto de test en este caso, sí se han identificado en la fase exploratoria y podrían influir en escenarios más generales.

```r
# Importancia de variables
importancia_rf=varImp(modelo_rf, scale = TRUE)

# Visualización
plot(importancia_rf, main = "Importancia de Variables Fisicoquímicas",
     col = "#973f5f", top = 10)
```



Las variables más importantes según Random Forest (intensidad del color, prolina, alcohol, flavonoides y OD280/OD315) coinciden con aquellas que presentan mayor contribución en el PCA, especialmente en la primera componente principal.